Faculty of Engineering

International Credit Hours Engineering Programs

Communication Systems Engineering Program

Academic Year 2025/2026–Fall 2025

ECE413
ASIC Design and Automation

Project 2:

PNR Back-End Design

Technical Report

Name	ID	E-mail
Mark Maher Eweida	21P0355	21P0355@eng.asu.edu.eg
Abdelhameed Mahmoud Sayed	21P0171	21P0171@eng.asu.edu.eg
Abdullah Ahmed Youssef	2100369	2100369@eng.asu.edu.eg

Table of Contents

1 Introduction

1.1 Project Objective

The primary objective of this project is to practice and execute the complete back-end physical design flow for a MIPS16 processor. Utilizing the ICC tool suite, the goal is to transform the provided design into a complete, manufacturable chip layout that meets strict physical and performance constraints. Specifically, the final layout must achieve a chip area of less than or equal to 100,000 μm^2 and an operating frequency of at least 400 MHz, while maintaining clean setup/hold timing and a clean Design Rule Check (DRC) report.

1.2 Project Idea

The core idea of this project involves optimizing a standard ASIC design flow to meet specific Quality of Results (QoR) targets. Starting with a provided reference script and the MIPS16 design, the methodology requires systematically applying and refining each stage of the back-end process, including floorplanning, power planning, placement, Clock Tree Synthesis (CTS), and routing. Beyond merely meeting the mandatory constraints, the design approach focuses on iterative optimization to close timing and physical verification, with an additional aim to minimize the area-delay product (area/frequency) for superior efficiency.

2 Steps of Back-End Design

2.1 Synthesis

Before physical design, RTL synthesis was performed using the provided synthesis scripts. The synthesis stage generates the gate-level netlist and the timing constraint file (SDC), which are then used as inputs for ICC. In our flow, synthesis is driven by syn.tcl and uses cons.tcl to define timing constraints (clock period, I/O delays, clock uncertainty, and hold false-path constraints).

2.1.1 Synthesis Setup, Link and Compile

```
set design mips_16
#set top_module mips_16_TOP

set_svf /home/ahesham/ref_flow_before/syn/output/${design}.svf

set_app_var search_path
"/home/ahesham/ref_flow_before/standard_cell_libraries

/NangateOpenCellLibrary_PDKv1_3_v2010_12/lib/Front_End
                        /Liberty/NLDM"
set_app_var link_library "* NangateOpenCellLibrary_ss0p95vn40c.db"
set_app_var target_library "NangateOpenCellLibrary_ss0p95vn40c.db"

sh rm -rf work
sh mkdir -p work
define_design_lib work -path ./work

analyze -library work -format verilog
/home/ahesham/ref_flow_before/rtl/${design}.v
elaborate $design -lib work
current_design

check_design
source /home/ahesham/ref_flow_before/syn/cons/cons.tcl
link
compile -area_effort high -map_effort high
```

This section prepares the synthesis environment, loads libraries, analyzes RTL, elaborates the top module, applies constraints, links the design.

The command compile -area_effort high -map_effort high runs the main synthesis optimization and technology mapping step. It maps the RTL into standard cells while trying to meet the timing constraints, and with high effort it explores more alternatives to reduce area and improve overall QoR compared to default settings.

2.1.2 Constraints

```
create_clock -name clk -period 2.5 -waveform {0 0.625} [get_ports clk]
set_max_area 0
set_input_delay -max 0.2 -clock [get_clocks clk] [remove_from_collection [all_inputs]
[get_ports clk]]
set_output_delay -max 0.2 -clock [get_clocks clk] [all_outputs]
set_clock_uncertainty 0.1 [get_clocks]
set_false_path -hold -from [remove_from_collection [all_inputs] [get_ports clk]]
set_false_path -hold -to [all_outputs]
```

These constraints define the STA environment: a primary clock clk with T = 2.5 ns (400 MHz) and a specified waveform, max I/O delays of 0.2 ns relative to the clock to model the external interface, and 0.1 ns clock uncertainty to cover skew/jitter margin. Hold checks on I/O boundaries are disabled using set_false_path -hold so hold closure targets internal sequential paths rather than unrealistic external hold requirements.

Constraining the area to achieve the maximum optimization.

2.1.3 Reporting

```
report_area > /home/ahesham/ref_flow_before/syn/report/synth_area.rpt
report_cell > /home/ahesham/ref_flow_before/syn/report/synth_cells.rpt
report_qor  > /home/ahesham/ref_flow_before/syn/report/synth_qor.rpt
report_resources > /home/ahesham/ref_flow_before/syn/report/synth_resources.rpt
report_timing -max_paths 10 > /home/ahesham/ref_flow_before/syn/report/synth_timing.rpt
```

This section presents the generation of synthesis reports, which are used to analyze the synthesis results, verify whether the design meets the required specifications, and identify any timing, area, or constraint violations.

2.1.4 Synthesis Output Generation and Netlist Cleanup

```
set_svf -off

write_sdc  /home/ahesham/ref_flow_before/syn/output/${design}.sdc

define_name_rules  no_case -case_insensitive
change_names -rule no_case -hierarchy
change_names -rule verilog -hierarchy
set verilogout_no_tri    true
set verilogout_equation  false

write -hierarchy -format verilog -output
/home/ahesham/ref_flow_before/syn/output/${design}.v
write -f ddc -hierarchy -output
/home/ahesham/ref_flow_before/syn/output/${design}.ddc
```

This section generates the final synthesis outputs and prepares the design for downstream physical implementation. The active timing constraints are exported as an SDC file to maintain consistency between synthesis and place-and-route. Design object names are normalized and converted to Verilog-compliant identifiers to avoid naming conflicts and tool compatibility issues. Verilog output options are configured to prevent the use of tri-state nets and equation-style expressions. Finally, the hierarchical gate-level Verilog netlist and the Design Compiler database (DDC) are written for use in physical design and potential future ECO iterations.

2.1.5 Synthesis Output

The synthesis process generates critical output files that serve as inputs to the place-and-route flow, most notably the SDC file, SVF file, and the synthesized netlist.

2.2 Design Setup and Import

```
set design mips_16
sh rm -rf $design
set sc_dir "/home/standard_cell_libraries/NangateOpenCellLibrary_PDKv1_3_v2010_12"
set_app_var search_path
"/home/standard_cell_libraries/NangateOpenCellLibrary_PDKv1_3_v2010_12
                        /lib/Front_End/Liberty/NLDM
                        \
                        /home/mohamed/Desktop/johnson/rtl"
set_app_var link_library "* NangateOpenCellLibrary_ss0p95vn40c.db"
set_app_var target_library "NangateOpenCellLibrary_ss0p95vn40c.db"
create_mw_lib   ./${design} \
                -technology $sc_dir/tech/techfile/milkyway/FreePDK45_10m.tf \
        -mw_reference_library $sc_dir/lib/Back_End/mdb \
        -hier_separator {/} \
        -bus_naming_style {[%d]} \
        -open
set tlupmax "$sc_dir/tech/rcxt/FreePDK45_10m_Cmax.tlup"
set tlupmin "$sc_dir/tech/rcxt/FreePDK45_10m_Cmin.tlup"
set tech2itf "$sc_dir/tech/rcxt/FreePDK45_10m.map"
set_tlu_plus_files -max_tluplus $tlupmax \
                   -min_tluplus $tlupmin \
               -tech2itf_map $tech2itf
import_designs  ../syn/output/${design}.v \
                -format verilog \
        -top ${design} \
        -cel ${design}
source  ../syn/cons/cons.tcl
set_propagated_clock [get_clocks clk]
save_mw_cel -as ${design}_1_imported
```

The ICC flow starts by setting up the design environment. First, a Milkyway library is created for the FreePDK45 technology, and the Nangate standard-cell timing library is loaded so ICC knows how fast each cell works. Then, the min/max RC (TLU+) files are set to estimate wire delays accurately. After that, the synthesized Verilog netlist is loaded as the top design mips_16, and the timing constraints from the SDC file are applied.

2.3 Floorplanning

```
create_floorplan -core_utilization 0.45 \
    -start_first_row -flip_first_row \
    -left_io2core 12.4 -bottom_io2core 12.4 -right_io2core 12.4 -top_io2core 12.4

report_ignored_layers
remove_ignored_layers -all
set_ignored_layers -max_routing_layer metal6

create_fp_placement

route_fp_proto -congestion_map_only -effort medium

extract_rc

report_timing -nosplit
report_timing -nosplit -delay_type min

report_constraint -all_violators -nosplit -max_delay
report_constraint -all_violators -nosplit -min_delay

save_mw_cel -as ${design}_2_fp
```

This part creates the initial floorplan for the design. The core utilization is set to 0.45 (45%) to leave enough whitespace for routing and reduce the risk of congestion later in the flow. Row options such as starting the first row and flipping alternate rows are enabled to keep standard-cell placement well-structured and friendly for routing. The I/O-to-core spacing is set to 12.4 µm on all sides to reserve margin around the core for routing access and for power structures added in later stages.

```
UTILIZATION RATIOS
------------------
Chip area           : 95308.04
Core area           : 80781.01
  SiteRow area      : 80781.01
Cell/Core Ratio     : 98.6499%
Cell/Chip Ratio     : 83.6135%
(A) Logical cell sites (non-fixed) :       149047
(B) Logical cell sites (fixed) :         2360
(C) All blocked sites :         6460
(D) Total core sites :       303688
Cell Utilization(non-fixed) = 50.15% (A) / (D - C)
                                           (149047) / (303688 - 6460)
Cell Utilization(non-fixed + fixed) = 50.54% (A + B) / (D - (C - B))
                                           (149047 + 2360) / (303688 - (6460 - 2360))
Blockage Percentage = 2.13% (C) / (D)
                                           (6460) / (303688)

DNET OPTIONS INFORMATION
```

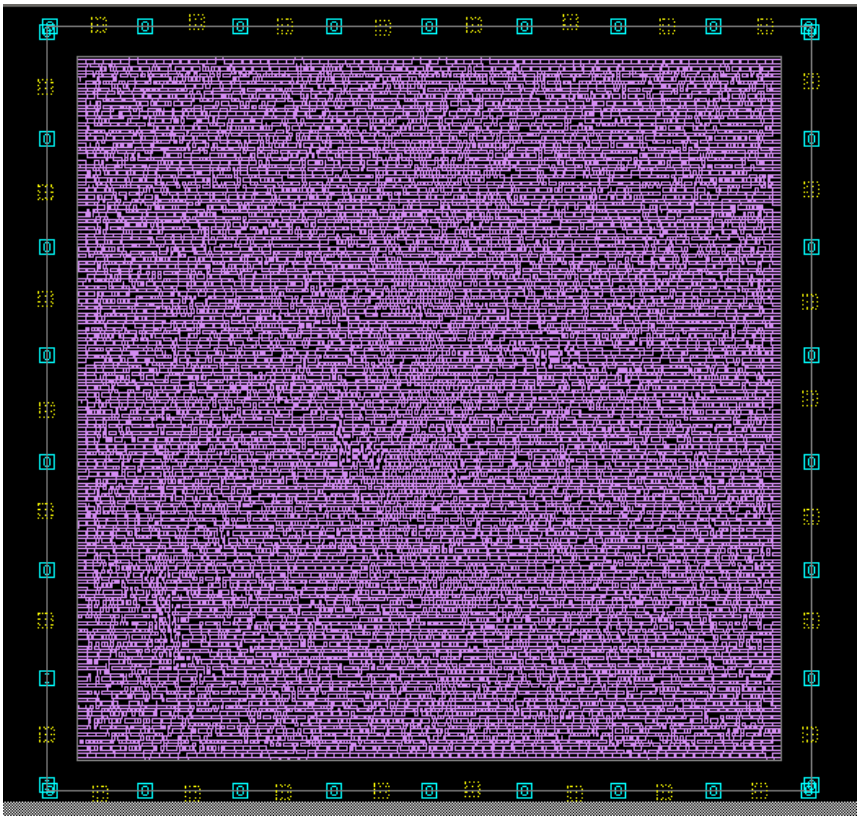
2.4 Power Planning

```
derive_pg_connection -power_net VDD \

                     -ground_net VSS \
                     -power_pin VDD \
                     -ground_pin VSS
set_fp_rail_constraints -set_ring -nets {VDD VSS} \
    -horizontal_ring_layer {metal7 metal9} \
    -vertical_ring_layer {metal8 metal10} \
    -ring_spacing 0.8 \
    -ring_width 5 \
    -ring_offset 0.8 \
    -extend_strap core_ring
set_fp_rail_constraints -add_layer -layer metal10 -direction vertical   -max_strap 128 -min_strap 10 -
min_width 2 -spacing minimum
set_fp_rail_constraints -add_layer -layer metal9  -direction horizontal -max_strap 128 -min_strap 10 -
min_width 2 -spacing minimum
set_fp_rail_constraints -add_layer -layer metal8  -direction vertical   -max_strap 128 -min_strap 10 -
min_width 2 -spacing minimum
set_fp_rail_constraints -add_layer -layer metal7  -direction horizontal -max_strap 128 -min_strap 10 -
min_width 2 -spacing minimum
set_fp_rail_constraints -add_layer -layer metal6  -direction vertical   -max_strap 128 -min_strap 10 -
min_width 2 -spacing minimum
set_fp_rail_constraints -set_global
create_fp_virtual_pad -net VDD -point {20.9685 309.2825}
create_fp_virtual_pad -net VDD -point {96.1045 308.7000}
create_fp_virtual_pad -net VDD -point {172.4060 309.2825}
create_fp_virtual_pad -net VDD -point {250.4545 308.7000}
create_fp_virtual_pad -net VDD -point {308.7000 286.5670}
create_fp_virtual_pad -net VDD -point {308.1175 197.4515}
create_fp_virtual_pad -net VDD -point {308.7000 110.6660}
create_fp_virtual_pad -net VDD -point {308.1175 22.7155}
create_fp_virtual_pad -net VDD -point {249.2930 -0.5865}
create_fp_virtual_pad -net VDD -point {171.8260 0.5785}
create_fp_virtual_pad -net VDD -point {95.5235 -0.5865}
create_fp_virtual_pad -net VDD -point {21.5510 -1.1690}
create_fp_virtual_pad -net VDD -point {-0.5825 68.7265}
create_fp_virtual_pad -net VDD -point {0.0000 153.7655}
create_fp_virtual_pad -net VDD -point {-0.5825 241.7170}
create_fp_virtual_pad -net VSS -point {-0.5825 284.2365}
create_fp_virtual_pad -net VSS -point {60.5760 310.4475}
create_fp_virtual_pad -net VSS -point {135.7130 308.1175}
create_fp_virtual_pad -net VSS -point {211.4330 310.4475}
create_fp_virtual_pad -net VSS -point {290.0650 308.7000}
create_fp_virtual_pad -net VSS -point {308.7040 242.8820}
create_fp_virtual_pad -net VSS -point {308.1215 158.4255}
create_fp_virtual_pad -net VSS -point {308.7040 68.7265}
create_fp_virtual_pad -net VSS -point {287.7355 -0.0035}
create_fp_virtual_pad -net VSS -point {213.1805 -1.1685}
create_fp_virtual_pad -net VSS -point {135.1310 -0.0035}
create_fp_virtual_pad -net VSS -point {58.8285 -1.1685}
create_fp_virtual_pad -net VSS -point {0.0000 22.7120}
create_fp_virtual_pad -net VSS -point {-0.5825 198.6150}
create_fp_virtual_pad -net VSS -point {-0.5825 112.9935}
synthesize_fp_rail -nets {VDD VSS} -synthesize_power_plan -target_voltage_drop 22 -voltage_supply 1.1 -
power_budget 500
commit_fp_rail
set_preroute_drc_strategy -max_layer metal6
preroute_standard_cells -fill_empty_rows -remove_floating_piece
analyze_fp_rail -nets {VDD VSS} -power_budget 500 -voltage_supply 1.1
create_fp_placement -incremental all
add_tap_cell_array -master TAP \
    -distance 30 \
    -pattern stagger_every_other_row
save_mw_cel -as ${design}_3_power
```

This section builds the power distribution network. Logical connections for VDD and VSS are derived, followed by the creation of power rings and a multi-layer power mesh across upper metal layers. Virtual power pads are added around the chip to model power entry points. The power network is synthesized with IR-drop constraints (targeting a maximum 2% voltage drop), committed, and analyzed. Standard cells are prerouted to power rails, and tap cells are inserted to ensure proper well biasing. The updated power-aware floorplan is saved.

We updated the virtual PG pad locations using the GUI compared to the earlier script to better match the final floorplan size and the power grid structure. In the original version, the pads were fewer and concentrated around specific edges, which can lead to uneven current entry and higher IR-drop in some regions. In the updated version, we redistributed the VDD and VSS virtual pads more uniformly around the chip boundary (top, bottom, left, and right), providing multiple injection points for the power mesh. This improves power delivery robustness, reduces local voltage drop risk, and helps the PDN meet the target IR-drop constraint more reliably during synthesize_fp_rail and analyze_fp_rail.



2.5 Placement

```
puts "start_place"
report_ignored_layers
check_physical_design -stage pre_place_opt
check_physical_constraints
place_opt -area_recovery
psynopt -area_recovery
check_legality
derive_pg_connection -power_net VDD \
                     -ground_net VSS \
                     -power_pin VDD \
                     -ground_pin VSS

set tie_pins [get_pins -all -filter "constant_value == 0 || constant_value == 0 && name
!~ V* && is_hierarchical == false "]
derive_pg_connection -power_net VDD \
                     -ground_net VSS \
                     -tie
connect_tie_cells -objects $tie_pins \
                  -obj_type port_inst \
                  -tie_low_lib_cell LOGIC0_X1 \
                  -tie_high_lib_cell LOGIC1_X1
puts "finish_place"
save_mw_cel -as ${design}_4_placed
```

Here, standard cells are placed and optimized. Pre-placement checks verify physical and constraint correctness. Initial placement is performed with area recovery enabled, followed by incremental optimization using psynopt. Timing legality and congestion are checked, and power/ground connections are re-derived. Tie cells are inserted to handle constant logic values. This stage produces a legalized and optimized placed design.

2.6 Clock Tree Synthesis (CTS)

```
puts "start_cts"
check_physical_design -stage pre_clock_opt
check_clock_tree
report_clock_tree
set_driving_cell -lib_cell BUF_X16 -pin Z [get_ports clk]
set_clock_tree_options \
    -clock_trees clk \
    -target_early_delay 0.1 \
    -target_skew 0.5 \
    -max_capacitance 300 \
    -max_fanout 10 \
    -max_transition 0.3
set_clock_tree_options -clock_trees clk \
    -buffer_relocation true \
    -buffer_sizing true \
    -gate_relocation true \
    -gate_sizing true
set_clock_tree_references -references [get_lib_cells */CLKBUF*]
define_routing_rule my_route_rule \
    -widths   {metal3 0.14 metal4 0.28 metal5 0.28} \
    -spacings {metal3 0.14 metal4 0.28 metal5 0.28}
set_clock_tree_options -clock_trees clk \
    -routing_rule my_route_rule \
    -layer_list "metal3 metal4 metal5"
set_clock_tree_options -use_default_routing_for_sinks 1
report_clock_tree -settings
clock_opt -only_cts -no_clock_route
report_design_physical -utilization
report_clock_tree -summary
report_clock_tree
report_clock_timing -type summary
report_timing
report_timing -delay_type min
report_constraints -all_violators -max_delay -min_delay
clock_opt -only_psyn -no_clock_route
route_group -all_clock_nets
derive_pg_connection -power_net VDD \
                     -ground_net VSS \
                     -power_pin VDD \
                     -ground_pin VSS
save_mw_cel -as ${design}_5_cts
puts "finish_cts"
```

CTS distributes the clock signal across the design. Clock checks are performed, and clock tree targets such as skew, transition, capacitance, and fanout are defined. Suitable clock buffer cells are selected, and a non-default routing rule (NDR) is applied to clock nets for better signal quality. The clock tree is synthesized, optimized, and routed in stages (CTS, CTO, clock routing). Timing and clock quality reports are generated, and the CTS-complete design is saved.

2.7 Routing and Post-Rout Optimization

```
puts "start_route"
check_physical_design -stage pre_route_opt
all_ideal_nets
all_high_fanout -nets -threshold 100
check_routeability
set_delay_calculation_options -arnoldi_effort low
set_route_options -groute_timing_driven true \
                  -groute_incremental true \
                  -track_assign_timing_driven true \
                  -same_net_notch check_and_fix
set_si_options -route_xtalk_prevention true \
               -delta_delay true \
               -min_delta_delay true \
               -static_noise true \
               -timing_window true
set_fix_hold [all_clocks]
set_prefer -min [get_lib_cells "*/BUF_X2 */BUF_X1"]
set_fix_hold_options -preferred_buffer
route_opt
psynopt -only_hold_time -congestion
route_zrt_eco -open_net_driven true
verify_zrt_route
route_zrt_detail -incremental true -initial_drc_from_input true
derive_pg_connection -power_net VDD \
                     -ground_net VSS \
                     -power_pin VDD \
                     -ground_pin VSS
save_mw_cel -as ${design}_6_routed
puts "finish route"
```

Before routing, spare cells are inserted to support future ECOs. Routing checks are performed, and timing-driven global and detailed routing is enabled. Signal integrity options such as crosstalk prevention and delta delay analysis are activated. Hold timing is fixed using preferred small buffers, and routing optimization is performed. ECO routing and final route verification ensure clean connectivity. The routed design is then saved.

2.7.1 Solving DRC

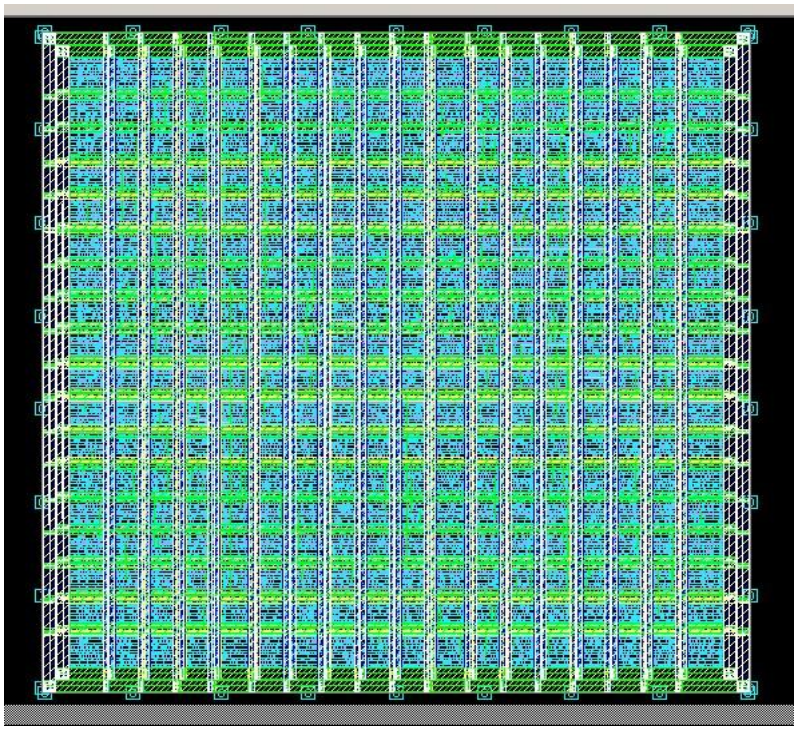
When we finished routing, we found 1 DRC caused by a wire that cut through a power sheet.

So we removed the wire then we rerouted this open using “route_zrt_eco -open_net_driven true”

2.8 Finishing and Generated Output Files

```
insert_stdcell_filler -cell_without_metal {FILLCELL_X32 FILLCELL_X16 FILLCELL_X8
FILLCELL_X4 FILLCELL_X2 FILLCELL_X1} \
    -connect_to_power VDD -connect_to_ground VSS
insert_zrt_redundant_vias
derive_pg_connection -power_net VDD \
                     -ground_net VSS \
                     -power_pin VDD \
                     -ground_pin VSS
save_mw_cel -as ${design}_7_finished
save_mw_cel -as ${design}
verify_zrt_route
verify_lvs -ignore_floating_port -ignore_floating_net \
           -check_open_locator -check_short_locator
set_write_stream_options -map_layer $sc_dir/tech/strmout/FreePDK45_10m_gdsout.map \
                         -output_filling fill \
                         -child_depth 20 \
                         -output_outdated_fill \
                         -output_pin {text geometry}
write_stream -lib $design \
             -format gds \
             -cells $design \
             ./output/${design}.gds
define_name_rules no_case -case_insensitive
change_names -rule no_case -hierarchy
change_names -rule verilog -hierarchy
set verilogout_no_tri true
set verilogout_equation false
write_verilog -pg -no_physical_only_cells ./output/${design}_icc.v
write_verilog -no_physical_only_cells ./output/${design}_icc_nopg.v
extract_rc
write_parasitics -output {./output/mips_16.spef}
close_mw_cel
close_mw_lib
exit
```

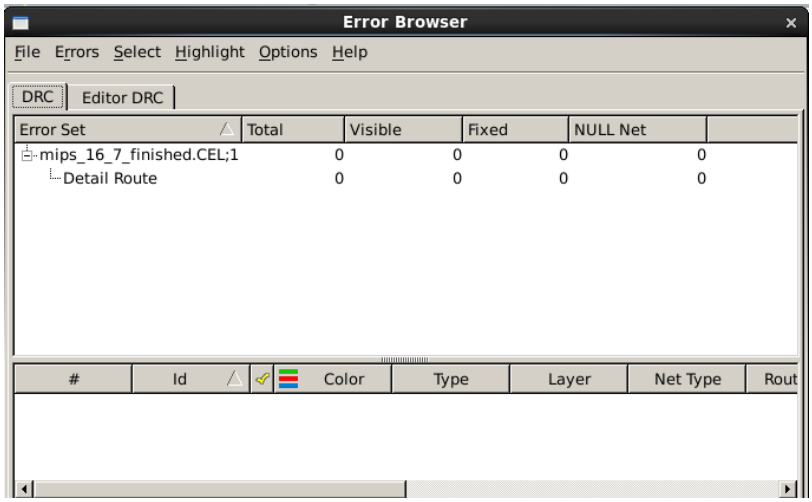
In the finishing stage, filler cells are inserted to meet density and spacing rules while maintaining power continuity. Redundant vias are added to improve reliability and manufacturability. Power and ground connections are verified again, and the final physical design state is saved.



2.8.1 Sign Off

Final timing verification was completed after RC extraction using RealRC with min/max parasitics. With a target clock period of 2.5 ns (400 MHz), the design satisfies both setup and hold requirements. The sign-off DRC run is clean with zero violations.

- Setup (max): worst slack = +1.80 ns (MET)
- Hold (min): worst slack = +0.00 ns (MET)
- Constraints: no violated constraints reported
- DRC: total violations = 0
- Routing statistics: wire length = 363,743 µm; contacts = 180,323; redundant-via conversion = 95.47%



Final sign-off checks are executed, including routing verification and LVS. The final GDS layout is generated for fabrication. Netlist names are cleaned and made Verilog-compliant, and post-layout Verilog netlists (with and without power/ground) are written. RC extraction is performed, and a SPEF file is generated for accurate post-layout timing analysis. Finally, the Milkyway library is closed and the flow exits.

3 Conclusion

3.1 Trials

- First, the design was optimized for timing, and the target frequency was successfully achieved; however, post-implementation analysis showed a total area of 166,000, far beyond the area constraint.

- An initial attempt was made to reduce the chip area by constraining the core utilization to 25%, but this did not bring the area within the required limit.

- The utilization was then increased to 36%, which led to a noticeable reduction in area, yet the design still failed to meet the area target.

- Finally, by further adjusting the floorplan and allowing a core utilization of 45%, the area was reduced sufficiently to satisfy the specified area requirement while preserving the desired performance

3.2 Summary of Results

The physical implementation of the MIPS16 processor has successfully met all target specifications outlined in the project requirements. The back-end design flow was executed using ICC, resulting in a final manufacturable layout with the following key metrics:

- Area: The final chip area is 95,308.04 μm^2 successfully remaining below the 100,000 μm^2 required.
- Frequency: The design is verified to operate at 400 MHz (2.5 ns clock period), meeting the minimum performance requirement5.
- Physical Verification: The routed design is DRC-clean with zero violations reported6.
- Bonus Metric: The achieved area-delay product is 238.27 μm^2/MHz.

3.3 Timing Analysis

Timing closure was achieved for both setup and hold paths. The design demonstrates a robust Setup Slack of +1.80 ns. Given the 2.5 ns clock period, this substantial positive slack indicates that the design has significant timing headroom and could theoretically operate at a higher frequency than the requested 400 MHz.

Conversely, the Hold Slack is +0.00 ns. While this satisfies the requirement for a clean report, it represents a zero-margin pass. This indicates that the hold constraints are tightly met, likely due to optimization efforts to fix hold violations during the routing or post-route stages.

3.4 Final Thoughts

The final MIPS16 layout meets the project requirements. The final chip area is within the 100,000 μm² constraint, and the design operates at 400 MHz based on a 2.50 ns clock period. Final timing reports show positive setup slack (+1.80 ns) and hold slack (+0.00 ns), and the final routing DRC check reports zero violations.

Further improvement opportunities include increasing hold margin above 0.00 ns and exploring alternative floorplan utilization and CTS cell choices to reduce the area–delay product.

3.5 Project Summary

This report documents the complete back-end design flow of a MIPS16 processor, from design import through floorplanning, power planning, placement, clock tree synthesis, routing, and sign-off checks. The final layout meets the required specifications of the project handout.

Requirement	Target	Achieved (from reports)	Status
Area	100,000 μm²	95,308.04 μm² (chip area)	PASS
Frequency	400 MHz	400 MHz (clock period = 2.5 ns)	PASS
Setup timing	Clean	Worst slack = +1.80 ns (MET)	PASS
Hold timing	Clean	Worst slack = +0.00 ns (MET)	PASS
DRC	Clean	Total violations = 0	PASS

Area-Delay Metric: area/frequency = 95,308.04 / 400 = 238.27 μm²/MHz.